

Java Backend Architecture

Interview Series

Chapter 10.7

Spring Data Elasticsearch

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

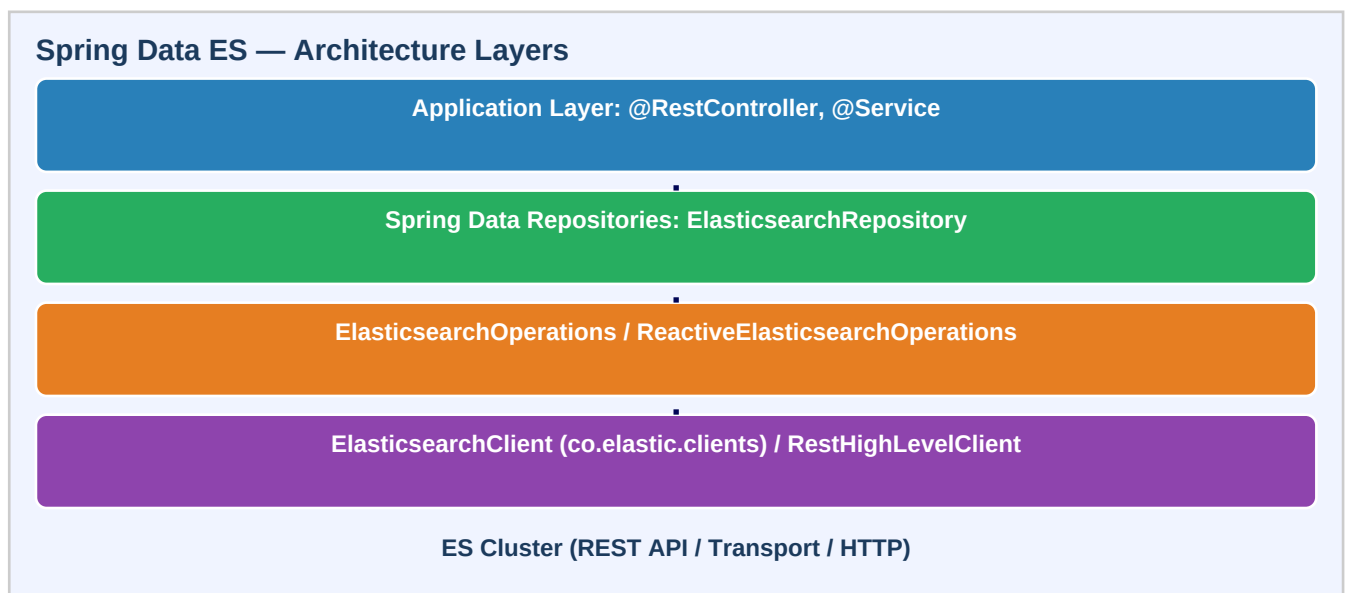
Chapter 10.7 – Spring Data Elasticsearch

Introduction

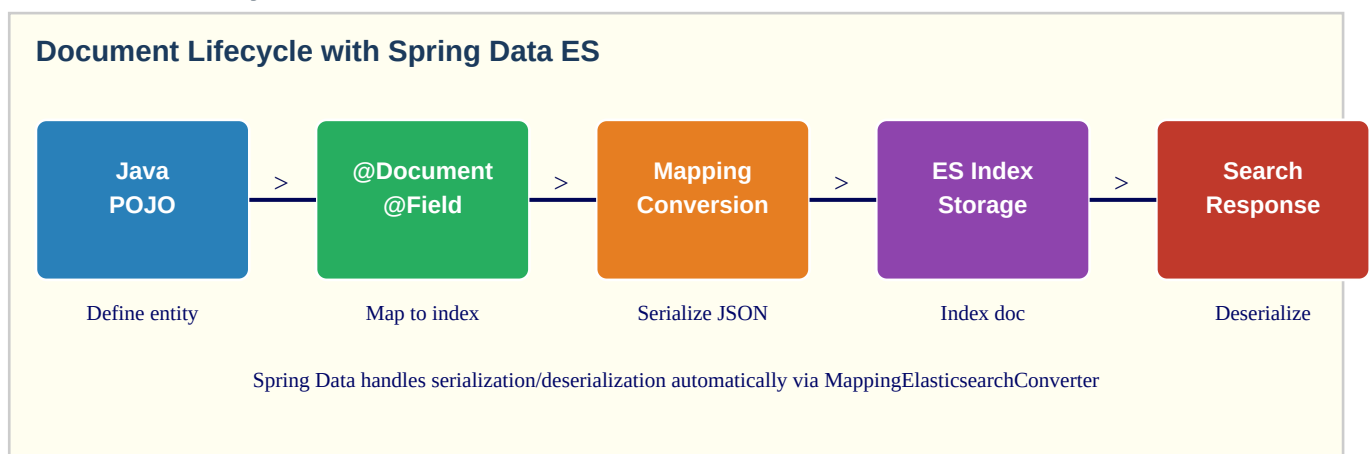
Spring Data Elasticsearch (SDE) integrates Elasticsearch into Spring-based Java applications using the familiar Spring Data repository abstraction. It provides entity mapping via annotations (`@Document`, `@Field`), repository interfaces with derived query methods, and the `ElasticsearchOperations` template for lower-level control. Starting with ES 8.x and SDE 5.x, the underlying client is the new `co.elastic.clients.ElasticsearchClient` (replacing the deprecated `RestHighLevelClient`).

SDE abstracts many boilerplate concerns — mapping, serialization, pagination — while still allowing access to the full Query DSL through `NativeQuery`. The reactive variant (`ReactiveElasticsearchOperations`) enables non-blocking ES access in WebFlux-based services.

Spring Data ES Architecture Layers



Document Lifecycle



Imperative vs Reactive Client

Imperative vs Reactive Elasticsearch Client

Imperative (Blocking)

- ElasticsearchOperations
- ElasticsearchRepository
- Returns T, List<T>, Page<T>
- Blocks thread per request

Reactive (Non-blocking)

- ReactiveElasticsearchOperations
- ReactiveElasticsearchRepository
- Returns Mono<T>, Flux<T>
- Uses Netty event loop

Use reactive for high-concurrency, low-latency services (WebFlux stacks)

Key Definitions

@Document	Marks a class as an ES document; specifies indexName and optional settings
@Field	Maps a Java field to an ES field with type, analyzer, and index options
@Id	Marks the ES document _id field
ElasticsearchRepository	Spring Data repository interface with CRUD + custom query methods
ElasticsearchOperations	Template class for full Query DSL access (imperative)
ReactiveElasticsearchOps	Non-blocking version returning Mono/Flux (WebFlux compatible)
NativeQuery	Wraps a Query DSL query for use in ElasticsearchOperations
CriteriaQuery	Object-oriented query builder without raw JSON
StringQuery	Raw JSON string query wrapped as a Spring Data query
Page	Paginated result container with content, totalElements, totalPages
SearchHit	Wraps a search result with score, highlight, explanation
SearchHits	Collection of SearchHit plus total hits, aggregations, suggest
IndexCoordinates	Runtime specification of index name(s) to query
@Query	Annotation on repository method specifying raw ES query as string
MappingElasticsearchConverter	Converts between Java POJOs and ES JSON documents

Version Compatibility Reference

Spring Data ES	Spring Boot	ES Server	ES Client
5.x	3.x	8.x	co.elastic.clients (new Java client)

4.x	2.7.x	7.x	RestHighLevelClient
3.x	2.3.x-2.6.x	7.x	RestHighLevelClient

Code Examples

1. Maven dependencies (Spring Boot 3 / ES 8)

```

<!-- pom.xml --> <dependency> <groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-elasticsearch</artifactId> </dependency> <!--
application.yml --> spring: elasticsearch: uris: http://localhost:9200 username: elastic
password: ${ES_PASSWORD} socket-timeout: 30s connection-timeout: 5s

```

2. @Document entity mapping

```

import org.springframework.data.annotation.Id; import
org.springframework.data.elasticsearch.annotations.*; @Document(indexName = "products",
createIndex = true) @Setting(settingPath = "/es/product-settings.json") // custom analyzers
public class ProductDocument { @Id private String id; @Field(type = FieldType.Text, analyzer =
"english", searchAnalyzer = "english") @MultiField(mainField = @Field(type = FieldType.Text,
analyzer = "english"), otherFields = { @InnerField(suffix = "keyword", type =
FieldType.Keyword, ignoreAbove = 256), @InnerField(suffix = "suggest", type = FieldType.Text,
analyzer = "autocomplete_index", searchAnalyzer = "autocomplete_search") }) private String
name; @Field(type = FieldType.Text, analyzer = "english", index = false, store = false) private
String description; @Field(type = FieldType.ScaledFloat, scalingFactor = 100) private
BigDecimal price; @Field(type = FieldType.Keyword) private String category; @Field(type =
FieldType.Keyword) private List<String> tags; @Field(type = FieldType.Boolean) private boolean
inStock; @Field(type = FieldType.Double) private double rating; @Field(type = FieldType.Date,
format = DateFormat.strict_date_optional_time) private Instant createdAt; // getters, setters,
builder omitted }

```

3. ElasticsearchRepository with derived methods

```

import org.springframework.data.elasticsearch.repository.ElasticsearchRepository; import
org.springframework.data.domain.*; public interface ProductRepository extends
ElasticsearchRepository<ProductDocument, String> { // Derived query – Spring Data builds the
query automatically List<ProductDocument> findByCategory(String category);
Page<ProductDocument> findByCategoryAndInStockTrue( String category, Pageable pageable);
List<ProductDocument> findByPriceBetween(BigDecimal min, BigDecimal max);
List<ProductDocument> findByTagsContaining(String tag); // @Query annotation – raw ES query as
JSON template @Query(""" { "bool": { "must": [ { "match": { "name": "?0" } } ], "filter": [ {
"term": { "category": "?1" } }, { "term": { "inStock": true } } ] } } """)
Page<ProductDocument> searchByNameAndCategory( String name, String category, Pageable
pageable); // Count query long countByCategory(String category); // Exists check boolean
existsByNameAndCategory(String name, String category); }

```

4. ElasticsearchOperations — full Query DSL control

```

@Service public class ProductSearchService { private final ElasticsearchOperations esOps;
public SearchHits<ProductDocument> complexSearch( String query, String category, BigDecimal
minPrice, BigDecimal maxPrice, int page, int size) { // Build query using NativeQuery + Query
DSL builder Query esQuery = NativeQuery.builder() .withQuery(q -> q .bool(b -> b .must(m ->
m .match(mt -> mt .field("name").query(query).fuzziness("AUTO"))) .filter(f -> f .term(t -> t
.field("category").value(category))) .filter(f -> f .range(r -> r .field("price")

```

```
.gte(JsonData.of(minPrice)) .lte(JsonData.of(maxPrice))) .filter(f -> f.term(t -> t
.field("inStock").value(true))) ) ) .withPageable(PageRequest.of(page, size, Sort.by(
Sort.Order.desc("_score"), Sort.Order.asc("price") ))) .withHighlightQuery(new HighlightQuery(
Highlight.builder() .withHighlighter(HighlightParameters.builder()
.withPreTags("<em>") .withPostTags("</em>") .withNumberOfFragments(3) .withFragmentSize(150)
.build()) .withField(new HighlightField("name")) .withField(new HighlightField("description"))
.build(), ProductDocument.class )) .build(); return esOps.search(esQuery,
ProductDocument.class); } // Access results public List<ProductWithHighlight> extractResults(
SearchHits<ProductDocument> hits) { return hits.stream() .map(hit -> new ProductWithHighlight(
hit.getContent(), hit.getHighlightField("name"), hit.getScore() ))
.collect(Collectors.toList()); } }
```

5. Aggregations via ElasticsearchOperations

```
public Map<String, Long> getCategoryCounts() { Query query = NativeQuery.builder() .withQuery(q
-> q.matchAll(m -> m)) .withAggregation("categories", Aggregation.of(a -> a .terms(t ->
t.field("category").size(20)) )) .withMaxResults(0) // hits not needed, only aggs .build();
SearchHits<ProductDocument> response = esOps.search( query, ProductDocument.class); // Extract
aggregation results ElasticsearchAggregations aggs = (ElasticsearchAggregations)
response.getAggregations(); return aggs.aggregationsAsMap() .get("categories") .sterms()
.buckets() .array() .stream() .collect(Collectors.toMap( bucket -> bucket.key().stringValue(),
StringTermsBucket::docCount )); }
```

6. Reactive Elasticsearch with WebFlux

```
@Repository public interface ReactiveProductRepository extends
ReactiveElasticsearchRepository<ProductDocument, String> { Flux<ProductDocument>
findByCategory(String category); Mono<Long> countByInStockTrue(); } @Service public class
ReactiveSearchService { private final ReactiveElasticsearchOperations reactiveOps; public
Flux<ProductDocument> searchReactive(String query, String category) { Query esQuery =
NativeQuery.builder() .withQuery(q -> q .bool(b -> b .must(m -> m.match(mt ->
mt.field("name").query(query))) .filter(f -> f.term(t -> t.field("category").value(category)))
) ) .withPageable(PageRequest.of(0, 20)) .build(); return reactiveOps.search(esQuery,
ProductDocument.class) .map(SearchHit::getContent) .onErrorResume(e -> { log.error("ES search
error", e); return Flux.empty(); }); } }
```

7. Index management with IndexOperations

```
@Component public class IndexManager { private final ElasticsearchOperations esOps; public void
ensureIndex() { IndexOperations indexOps = esOps.indexOps(ProductDocument.class); if
(!indexOps.exists()) { // Create with mapping from @Document/@Field annotations
indexOps.createWithMapping(); log.info("Created products index"); } else { // Update mapping
(only adds new fields – won't modify existing) Document mapping =
indexOps.createMapping(ProductDocument.class); indexOps.putMapping(mapping); } } public void
reindex(String newIndexName) { IndexOperations oldOps =
esOps.indexOps(IndexCoordinates.of("products_v1")); IndexOperations newOps =
esOps.indexOps(IndexCoordinates.of(newIndexName)); // Create new index newOps.create();
newOps.putMapping(newOps.createMapping(ProductDocument.class)); // Reindex ReindexRequest
reindexReq = ReindexRequest.builder( IndexCoordinates.of("products_v1"),
IndexCoordinates.of(newIndexName) ).build(); // Execute via low-level client for full control }
public void addAlias(String indexName, String aliasName) { IndexOperations ops =
esOps.indexOps(IndexCoordinates.of(indexName));
ops.addAlias(AliasData.of(AliasName.of(aliasName))); } }
```

Interview Questions & Answers

Q1. What is Spring Data Elasticsearch and how does it relate to the ES Java client?

Spring Data Elasticsearch (SDE) is the Spring Data module for ES. It provides entity mapping (@Document, @Field), repository abstraction (ElasticsearchRepository), and template operations (ElasticsearchOperations). Under the hood, SDE 5.x uses the new official Java client (co.elastic.clients.ElasticsearchClient) which communicates over HTTP REST. SDE adds the Spring Data conventions on top — derived queries, pagination, entity lifecycle — abstracting the raw client while still allowing pass-through to it.

Q2. What annotations are needed to map a Java class to an ES index?

@Document(indexName='...') marks the class as an ES entity and specifies the index name. @Id on the String id field maps to ES _id. @Field(type=FieldType.Text, analyzer='english') configures individual fields. @MultiField creates multi-field mappings (text + keyword sub-fields). @Setting(settingPath='/path.json') points to a JSON file with custom analyzers. If createIndex=true (default), SDE creates the index on application startup if it doesn't exist.

Q3. How do you write a custom query in a Spring Data ES repository?

Two approaches: (1) Derived method names — Spring Data parses the method name to build the query: findByNameAndCategory, findByPriceBetween, findByTagsContaining. Simple but limited. (2) @Query annotation — specify a raw ES Query DSL JSON string as the annotation value, with ?0, ?1 placeholders for method parameters. For complex queries needing NativeQuery/function_score/aggregations, inject ElasticsearchOperations and build the query programmatically.

Q4. What is the difference between ElasticsearchRepository and ElasticsearchOperations?

ElasticsearchRepository is the Spring Data repository interface — it provides derived query methods, @Query annotation support, and basic CRUD. It's convenient for simple queries. ElasticsearchOperations is the lower-level template — it exposes the full Query DSL via NativeQuery, CriteriaQuery, and StringQuery. Use it for: complex bool queries, aggregations, highlighting, script queries, reindexing, and index management. Internally, repositories delegate to ElasticsearchOperations.

Q5. Explain NativeQuery and when you would use it.

NativeQuery wraps a full ES Query DSL query (built with co.elastic.clients query builders) for use with ElasticsearchOperations. Use it when derived methods or @Query aren't sufficient: complex bool queries with multiple must/filter/should clauses, aggregations, highlighting, function_score, suggesters, or any advanced DSL feature. It's the escape hatch for full ES power within the Spring Data abstraction.

Q6. How does Spring Data Elasticsearch handle pagination?

Pass a Pageable to repository methods: findByCategory(cat, PageRequest.of(0, 20, Sort.by('price'))). Returns a Page with content, total elements, and page metadata. Internally, SDE translates Pageable into ES from/size. For deep pagination, use withPageable() in NativeQuery — but remember ES's 10k max_result_window limit. For deep pagination beyond 10k, use search_after with NativeQuery (no Spring Data abstraction for this — requires manual cursor management).

Q7. What is SearchHits vs Page in Spring Data ES?

Page is the standard Spring Data page — content list, total elements, pages. It loses ES-specific metadata like scores, highlights, and aggregations. SearchHits is ES-specific — it contains SearchHit objects (with score, highlight fragments, sort values, explanation) plus total hits, aggregations, and suggest response. Use SearchHits when you need highlights or aggregations; use Page when you only need the entities and

pagination info.

Q8. How do you access aggregation results from a search response?

Use `ElasticsearchOperations` with `NativeQuery` containing `.withAggregation()`. The result `SearchHits` provides `.getAggregations()` returning an `Aggregations` interface. Cast to `ElasticsearchAggregations` (SDE 5.x) and call `.aggregationsAsMap()` or `.get(name)`. Navigate the aggregation tree: `.sterms()` for string terms, `.dterms()` for double terms, `.dateHistogram()` for date histogram, etc. Each aggregation level can contain sub-aggregations.

Q9. How do you configure custom analyzers in Spring Data Elasticsearch?

Use `@Setting(settingPath="/classpath/es-settings.json")` on the `@Document` class. The JSON file follows the ES index settings format with `analysis.analyzer` and `analysis.filter` sections. SDE reads this file and applies it when creating the index (`createIndex=true`). Alternatively, programmatically use `IndexOperations.create(Map settings)` for full control. For per-field analyzer, use `@Field(analyzer='my_custom_analyzer')`.

Q10. Explain the difference between `@Field index=false` and not having the field in the entity.

`@Field(index=false)` explicitly tells ES not to index the field (not searchable) but still include it in the mapping and `_source`. The field is in the mapping schema. A field absent from the entity class won't be in the mapping if `dynamic=strict`, but if `dynamic=true`, ES might auto-map it. For fields you want to store but not search (e.g., raw HTML), use `index=false`. For fields that shouldn't exist in ES at all, use `@Transient`.

Q11. What is `@Transient` in Spring Data ES?

`@Transient` (from `spring-data-commons`) marks a Java field that should NOT be persisted to Elasticsearch. The field exists only in the Java object, never in the index mapping or stored document. Use it for: computed display fields populated after retrieval, fields from other data sources joined in Java, security-sensitive fields that shouldn't be indexed, or temporary state fields in the entity.

Q12. How does Spring Data ES handle index aliasing?

Use `IndexOperations.addAlias(AliasData.of(AliasName.of('products')))` to add an alias. The `@Document` annotation's `indexName` can be either the actual index name or an alias name — SDE resolves it at runtime. Best practice: always use an alias as the `indexName` in `@Document`. This enables zero-downtime reindexing: create new index, populate it, atomically swap alias from old to new via `IndexOperations`.

Q13. How do you handle index creation and mapping updates on application startup?

With `@Document(createIndex=true)`, SDE creates the index if absent. For mapping updates: inject `IndexOperations` on startup (`@EventListener(ApplicationReadyEvent.class)`) and call `indexOps.putMapping(indexOps.createMapping(YourClass.class))`. This adds new fields to the existing mapping. It does NOT change existing field types — those require reindexing. For production, manage mapping changes explicitly via migration scripts, not auto-create.

Q14. What is the `ReactiveElasticsearchOperations` and when would you use it?

`ReactiveElasticsearchOperations` is the non-blocking equivalent of `ElasticsearchOperations`, returning `Mono` and `Flux` instead of `T` and `List`. Use it in `WebFlux` (reactive) Spring applications where blocking I/O would starve the event loop. The reactive client uses `Netty` under the hood. For blocking MVC applications, use the imperative version — the reactive client has higher complexity without benefit in a blocking context.

Q15. How do you implement multi-index search in Spring Data ES?

Pass `IndexCoordinates.of('index1', 'index2')` to `ElasticsearchOperations.search(query, clazz, IndexCoordinates.of('products', 'products_archive'))`. For cross-index search with wildcard: `IndexCoordinates.of('products-*')`. Note: the entity class must have compatible mappings across all targeted indices. Cross-index responses may have score inconsistencies (different IDF per shard) — use `dfs_query_then_fetch` search type for consistent scoring (set via `SearchRequest` customization).

Q16. How do you write a Spring Data ES test?

Use `@DataElasticsearchTest` — sets up an embedded ES for testing without full application context. Or use `Testcontainers` with `@SpringBootTest`: start an ES container and configure `spring.elasticsearch.uris` to the container's address. Use `ElasticsearchOperations` or repository directly. Clean between tests with `indexOps.delete()`. For unit testing query logic without ES, mock `ElasticsearchOperations` with `Mockito` and verify search was called with the expected query.

Q17. What are the known limitations of Spring Data ES vs native ES client?

1) Aggregations API is lower-level — requires casting to `ElasticsearchAggregations` and navigating the typed tree. Less fluent than raw client. 2) `search_after` pagination not abstracted — requires `NativeQuery` with manual sort values. 3) Script queries, LTR plugin queries, async bulk — require dropping to raw client. 4) Slower to adopt new ES features — must wait for SDE version. 5) Annotation-based mapping can diverge from the actual index if migrations aren't managed. For these cases, inject the raw `ElasticsearchClient` bean and use it directly.

Q18. How do you configure connection pooling and timeout in SDE?

In Spring Boot, set `spring.elasticsearch.socket-timeout`, `connection-timeout`, and `spring.elasticsearch.restclient.sniffer.*` for automatic node discovery. For programmatic configuration, create a `ClientConfiguration` bean via `ClientConfiguration.builder().connectedTo('host:9200').withSocketTimeout(30000).withConnectTimeout(5000).build()`. The underlying HTTP client (Apache `HttpClient`) manages connection pooling — configure max connections via `RestClientBuilderCustomizer` bean.

Q19. How does Spring Data ES handle ID generation?

If the `@Id` field is null at index time, ES generates a UUID as the document ID. SDE populates the `@Id` field with the generated ID after indexing (via the index response). If you provide an ID, ES creates or replaces the document with that ID. Use deterministic IDs (e.g., UUID v5 from business key) to make indexing idempotent — re-indexing the same document with the same ID is an upsert (full replace), not a duplicate.

Q20. What is BulkOperations in Spring Data ES and how do you use it?

`BulkOperations` batches multiple index/update/delete operations into a single HTTP request. Get via `esOps.bulkOps(BulkMode.CONCURRENT, ProductDocument.class)`. Add operations: `.save(entity)`, `.update(query, updateDef)`, `.delete(query)`. Execute with `.execute()` returning `BulkResponse`. Check for failures via `bulkResponse.getItems()`. Use for batch indexing (e.g., full reindex), periodic sync from DB, or any case where you're indexing hundreds+ documents — dramatically reduces HTTP overhead.

Q21. How do you update a document without full replace in Spring Data ES?

Use `ElasticsearchOperations` with an `UpdateQuery` targeting specific fields. Create an `UpdateQuery` with: document (Map of fields to update), script update, or upsert. Example: `UpdateQuery.builder(id).withDocument(Document.create().append('price', 99.99)).build()`. Call `esOps.update(query, IndexCoordinates.of('products'))`. Partial updates use ES's update API (merge, not replace). For bulk partial updates: `BulkOperations.updateByQuery()` with a script.